

Human-in-the-Loop Legacy Code Refactoring with RAG-Grounded Hallucination Analysis

Karthik Alluri

Department of Computer Science
University of Colorado Boulder
kaal6482@colorado.edu

Abstract—Legacy Python codebases tend to drift over time into long functions, deep nesting, mutable globals, unsafe SQL patterns, and missing docstrings. Static analyzers flag these issues but do not fix them, and unconstrained LLM-based refactoring tends to introduce hallucinated calls, silent behavioral changes, and lost formatting. We present LCR (Legacy Code Refactor), a human-in-the-loop system that combines deterministic static analysis, dual plan generation, retrieval-augmented generation (RAG), lossless concrete syntax tree patching, and a six-stage QA layer. A mandatory approval gate forces a human reviewer to choose between two competing plans before any code is generated. Evaluated across 19 legacy Python projects producing 1,273 refactoring edits and 292 QA-validated runs, LCR achieves a 66.8% first-pass QA success rate with large per-function effect sizes (Cohen’s $d > 0.88$ for CC and LOC). The RAG feedback loop auto-ingested 17 validated refactorings as gold examples without manual curation, and smell analysis across 1,273 edits reveals that documentation debt and security-critical patterns account for over 44% of refactoring demand in legacy Python codebases.

Index Terms—Legacy code refactoring, human-in-the-loop, retrieval-augmented generation, hallucination, static analysis, code quality metrics.

I. INTRODUCTION

SOFTWARE maintenance dominates lifecycle cost [1], and most of that cost traces back to accumulated technical debt [2]. Over time, codebases drift into recognizable patterns: functions too long to read in one sitting, control flow nested too deep to trace, undocumented helpers, mutating globals, and SQL strings built by concatenation. These patterns degrade coupling and cohesion [3], hurt maintainability [4], and show up even in test code [5]. Static analyzers reliably *find* them [6]; what they reliably do not do is fix them.

LLMs look like the obvious way to close that gap [7], but they introduce a new class of problems in practice. Generated code routinely calls functions that do not exist, changes behavior silently, and strips comments and formatting — producing diffs visually larger than their semantic content and difficult to review [8]. Recent tools such as EM-Assist wrap LLM output in safety checks [9], and hybrid analysis-plus-LLM approaches [10] and learning-based detectors [11] have reduced these risks, but no existing system brings together human oversight at the strategy level, RAG grounding, lossless patching, multi-stage QA, and full audit logging under one roof.

LCR is built around a single design bet: the most consequential decision in an automated refactoring workflow is not what code to write, but which strategy to commit to. That decision belongs to a human. LCR enforces this through a mandatory approval gate that fires *after* two competing plans have been fully drafted but *before* any code is generated. Earlier in the workflow the reviewer has nothing concrete to compare; later, the reviewer is just rubber-stamping work that already cost an inference call.

Beyond the architecture, this paper reports operational evidence of the RAG feedback loop: across 292 QA-validated runs, 17 refactored functions were auto-ingested into the knowledge base as gold examples, all gated on full QA passage. A controlled three-condition ablation (RAG-enabled vs. RAG-disabled vs. random-KB) is identified as the primary direction for future work.

The contributions of this work are:

- **Dual-plan generation with an approval gate.** A deterministic plan from static analysis (Plan 1: AST+RAG) and an LLM-authored plan over the same retrieved context (Plan 2: LLM+RAG) are presented side-by-side; no

code is generated until a reviewer picks one.

- **Six-stage QA layer.** Compilation, regression tests, smell regression, undefined-call detection, mutation testing, and a bounded retry loop, each targeting a different failure mode.
- **Full audit trail.** SQLite (required) and Neo4j (optional) persist every plan, patch, metric snapshot, and validation outcome.
- **Empirical evaluation on 19 projects.** Real measurements from 292 QA-validated runs and 1,273 refactoring edits, with large effect sizes (Cohen’s $d > 0.88$) on per-function CC and LOC.
- **Smell distribution analysis.** First characterisation of smell category prevalence across legacy Python projects, revealing that security-critical smells account for 25.4% of all refactoring demand.

The remainder of this paper is organized as follows. Section II surveys related work across smell detection, LLM-driven refactoring, and RAG for code. Section III describes the LCR system architecture and the rationale behind each design decision. Section IV presents the experimental setup, research questions, and results. Section V discusses threats to validity, and Section VI concludes with directions for future work.

II. RELATED WORK

Automated refactoring sits at the intersection of three active research threads: smell detection and metric-driven repair, LLM- and bot-based code transformation, and retrieval-augmented generation for code synthesis. Prior work in each thread has contributed important building blocks, but no existing system assembles them into a single, end-to-end workflow with human oversight, RAG grounding, and multi-stage validation. The following subsections survey the most relevant efforts and locate the gap that LCR addresses.

A. Smell Detection and Metric-Driven Refactoring

Fowler’s catalog [1] remains the reference point for smell-driven refactoring, and three systematic reviews [2], [12], [13] converge on the same conclusion: no single transformation universally improves all quality attributes, and the gap between detection and automated repair remains the field’s most stubborn open problem. Earlier metric-driven

work [3], [4] showed that guided refactoring is tractable without full automation, but relies on hand-crafted prediction models and does not generate patches. Test-specific catalogs [5] and search-based optimization [14] address narrower slices of the problem. Pailoor et al. [15] introduced Revamp, which uses counterexample-guided synthesis for semantic ADT refactoring; it is rigorous but scoped to a narrow operation class. Together, these efforts establish the motivation for smell-driven repair without closing the loop from diagnosis to validated patch.

B. Automated, Bot-Based, and LLM-Driven Refactoring

Wyrich and Bogner [10] integrated static analysis with a GitHub PR workflow, but used a fixed transformation catalog without RAG or multi-stage validation. RefactoringMiner [6] detects refactorings retrospectively from commit history but does not generate them. Empirical studies of real refactoring commits [16], [17] reinforce the case for human oversight. On the learning side, ReFT5 [11] classifies refactoring types but does not produce patches; GNN-based AST methods [18] and hybrid neural smell detectors [19] improve detection but lack end-to-end validation. Few-shot LLM refactoring [7], EM-Assist [9], and exploratory studies of developer-ChatGPT conversations [20] show that LLMs can perform non-trivial refactorings, but Ampatzoglou et al. [8] report that unstructured LLM reasoning produces inconsistent results — the finding that motivates LCR’s QA layer. Refactoring has also been applied outside the maintainability setting [21], illustrating the breadth of the underlying toolkit. Mo et al. [22] explored a RL/GNN agent for human-LLM collaborative refactoring, but without strong empirical validation or RAG grounding. LCR places the human review step specifically at plan selection, after both candidate plans are fully elaborated.

C. Retrieval-Augmented Generation for Code

RAG was introduced by Lewis et al. [23] as a hallucination-reduction technique for knowledge-intensive NLP. In the code domain, Gao et al. [24] proposed the Retrieve–Refactor–Generate (RRG) framework, showing that refactoring the retrieved context itself improves generation quality, though RRG is positioned as a generation enhancement

rather than a full refactoring framework. Scalable vector infrastructure [25] makes repository-aware retrieval practical. To our knowledge, no prior work has evaluated RAG-grounded refactoring against a retrieval-disabled baseline using a reproducible hallucination proxy under a controlled three-condition ablation; closing this gap empirically is the primary direction for future work in this line of research.

III. SYSTEM ARCHITECTURE

LCR is designed around a core insight: the risks introduced by LLM-assisted refactoring — hallucinated symbols, silent behavioral changes, and unreviewed structural decisions — require deterministic guardrails at every stage rather than a single post-hoc safety check. This section describes how each component addresses a specific failure mode, and why the design choices were made in the way they were. LCR is implemented in Python 3.11 with both a CLI for scripted use and a Streamlit web app for interactive use. The Orchestrator runs the phases in order:

Repository → Inspector → Plan Gate → RAG Retrieval → Builder → QA → Artifacts

Figure 1 shows the end-to-end flow of the LCR system, from repository ingestion through to refactored code output.

A. Inspector (Static Analysis)

The Inspector is the foundation of the entire system: every downstream decision — which functions to target, which plans to generate, and which QA thresholds to enforce — depends on the accuracy and completeness of the static analysis it produces. Without a rich, multi-view representation of the codebase, plan generation would be uninformed guesswork and metric-based QA would have no reliable baseline to compare against.

The Inspector builds four views of the repository: an AST layer for function boundaries, nesting, argument counts, global mutations, docstrings, and return types; a per-function CFG for branching structure; a NetworkX call graph for cross-file impact and post-patch signature checks; and a lightweight type inference pass to cut false positives in context-sensitive smells. These feed a smell detector covering eight categories: `long_function`,

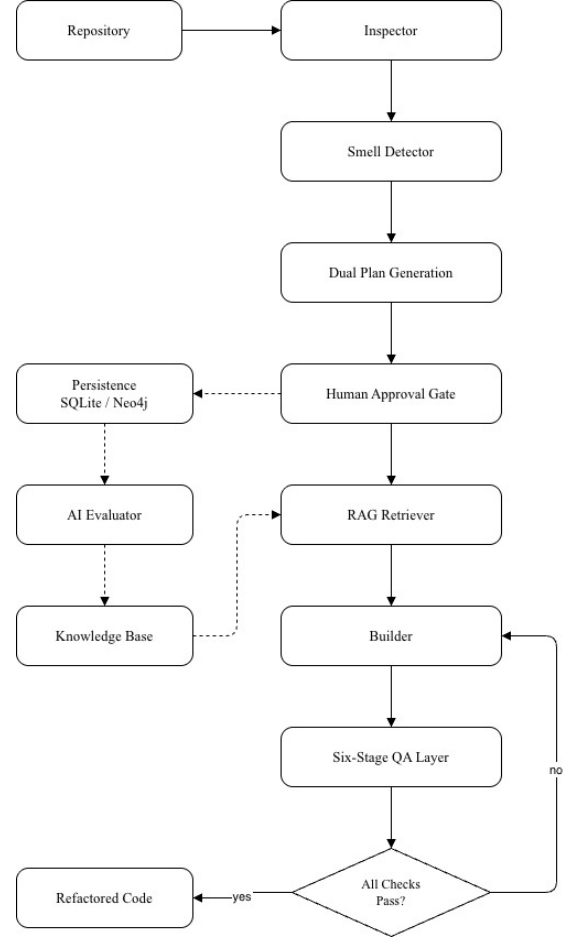


Fig. 1. Overall LCR workflow. The system flows from repository ingestion through static analysis, dual-plan generation, human approval, RAG retrieval, and patch generation, concluding with a multi-stage QA check. Dashed arrows indicate optional or feedback paths; the diamond marks the final pass/fail decision.

`deep_nesting`, `too_many_args`, `mutates_global`, `sql_injection_risk`, `query_in_loop`, `missing_docstring`, and `no_return_type`. The Inspector also records five per-function metrics — cyclomatic complexity (CC) [26], cognitive complexity, maintainability index (MI) [27], LOC, and CFG complexity — which both prioritize candidates and serve as the pre/post measurements used in the evaluation. Figure 2 illustrates the internal flow of the Inspector, from raw repository views through to smell detection and metric output.

B. Dual-Plan Generation and Approval Gate

The central challenge in applying LLMs to refactoring is that the most consequential decision — which strategy to pursue — is also the one most

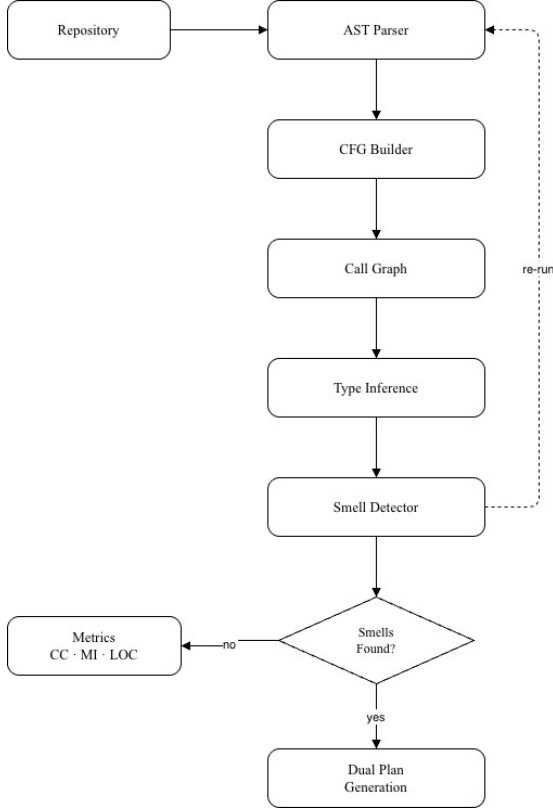


Fig. 2. Inspector and Smell Detector workflow. The repository is parsed into four complementary views — AST, CFG, call graph, and type inference — which converge into the Smell Detector. If smells are found the system proceeds to dual plan generation; otherwise metrics are recorded and the run terminates cleanly.

likely to be wrong if left entirely to an automated system. A rule-based plan is reproducible but rigid; an LLM-authored plan is creative but unpredictable. Rather than pick one, LCR generates both and requires a human to choose, ensuring that strategy selection is always accountable.

LCR generates two plans before any code is written. **Plan 1 (AST+RAG)** is built deterministically from smell detector output, with RAG providing grounding context but no LLM authorship; given the same code and index, it is fully reproducible. **Plan 2 (LLM+RAG)** uses an LLM over the same context to propose an alternative, often catching decompositions that rule-based logic would miss. Both plans share a common schema listing target functions, smell categories, transformation types, and expected metric impact.

The **approval gate** blocks every downstream phase until a reviewer picks one plan. The gate fires after both plans are fully drafted but before the Builder runs. Earlier, the reviewer would only see

raw smell data with no strategy to evaluate; later, they would only be ratifying code that has already cost an inference call. This placement aligns human judgment with the artifact — the approved plan — that controls the patch, the retrieval query, and the QA policy applied downstream. For controlled experiments, a pre-approved plan can be loaded programmatically.

C. RAG System

The primary failure mode of unconstrained LLM code generation is that the model invents plausible-sounding but nonexistent identifiers — functions, classes, and imports that do not exist in the target repository. The RAG system exists specifically to anchor generation to the actual codebase, reducing this hallucination risk by ensuring that retrieved context is both semantically relevant and identifier-specific.

The knowledge base contains documentation, language best-practice notes, and gold refactoring examples — including ones auto-ingested from prior validated runs. Documents are chunked and embedded with sentence-transformers, indexed in either FAISS [25] or MiniVectorDB. A hybrid retriever combines dense similarity with keyword ranking so that function identifiers and library names stay discoverable when embeddings alone fall short. A bounded context assembler caps the prompt size, and the approved plan steers the query so retrieved context aligns with the chosen strategy. Each successful run feeds its refactored functions back into the index, slowly raising retrieval quality without manual curation. The complete RAG retrieval flow is shown in Figure 3.

D. Builder and Lossless Patching

A common failure mode of LLM-generated code changes is that they appear correct in a diff but silently discard comments, docstrings, and formatting that carry significant semantic and review value. The Builder is therefore designed not just to generate a correct patch, but to apply it in a way that is auditable and merge-ready — which requires lossless concrete syntax tree manipulation rather than standard AST rewriting.

The Builder takes the approved plan and assembled context, builds a structured prompt, and calls a configured LLM (Anthropic Claude Sonnet,

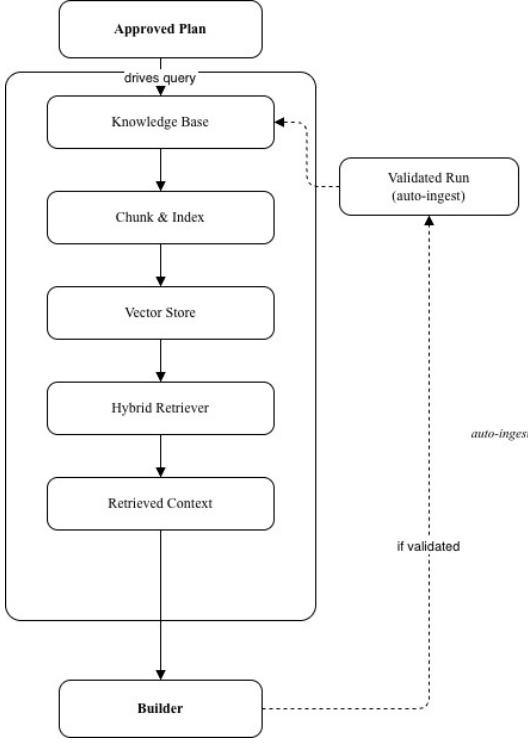


Fig. 3. RAG System workflow. The approved plan drives a query against the knowledge base, which is chunked, indexed into a vector store, and retrieved via a hybrid retriever. Validated runs are auto-ingested back into the knowledge base, gradually raising retrieval quality without manual curation.

OpenAI GPT-4o, Google Gemini, or any Ollama model). Patches are applied with `libcst` [28], which preserves comments, docstrings, and whitespace — the standard `ast` module discards all of these, producing diffs that look much larger than their semantic content and are difficult to review. A `touch_only` constraint is enforced at the file-system level so the LLM cannot edit anything outside the approved plan, and all changes go to a working copy until QA passes. Once QA is satisfied, results are persisted and scored by the AI Evaluator, as shown in Figure 4.

E. Quality Assurance Layer

No single validation check is sufficient to catch all the failure modes of LLM-assisted refactoring. Syntactic correctness does not guarantee behavioral equivalence; passing regression tests does not mean new smells were not introduced; and a clean-looking patch may still contain hallucinated symbol references. The six-stage QA layer addresses this by targeting a distinct failure mode at each step, in an

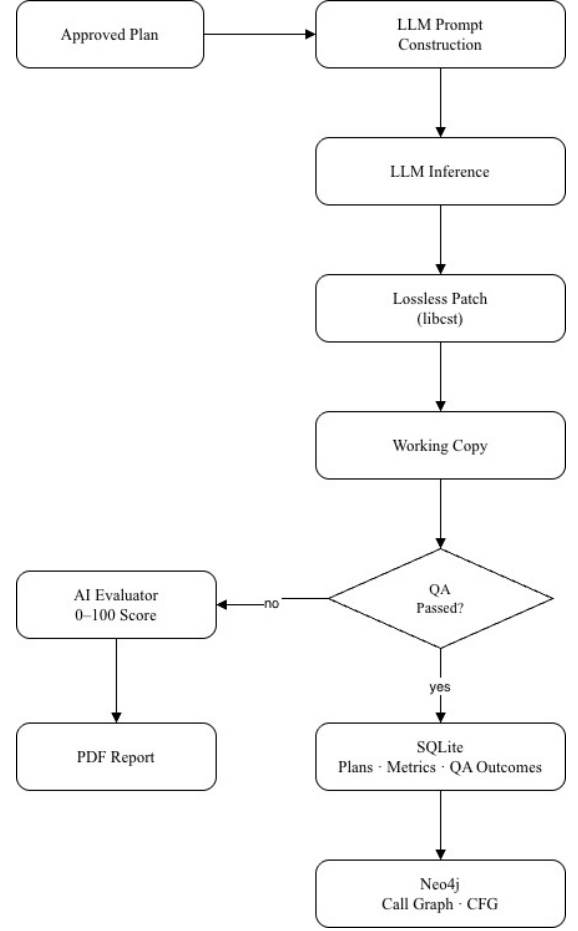


Fig. 4. Builder, Persistence, and AI Evaluator workflow. The approved plan is used to construct an LLM prompt, generate a lossless patch via `libcst`, and apply it to a working copy. On QA passage the outcome is persisted to SQLite and Neo4j; the AI Evaluator produces a 0–100 score and PDF report. On failure the run is routed to the AI Evaluator for diagnosis.

order that fails fast on the cheapest checks first. The complete QA flow is illustrated in Figure 5.

The QA layer runs six checks in order, each targeting a different failure mode:

- 1) **Compilation** (`py_compile`) for syntactic validity.
- 2) **Regression tests** (`pytest`) against the project’s existing suite.
- 3) **Smell regression**: re-run the Inspector and confirm no smell category got worse.
- 4) **Undefined-call detection**: AST pass flagging references to symbols not resolvable in scope. This is the proxy metric for hallucination in the evaluation.
- 5) **Mutation testing** (`mutmut` [29]) to check that the test suite meaningfully constrains the refactored code.

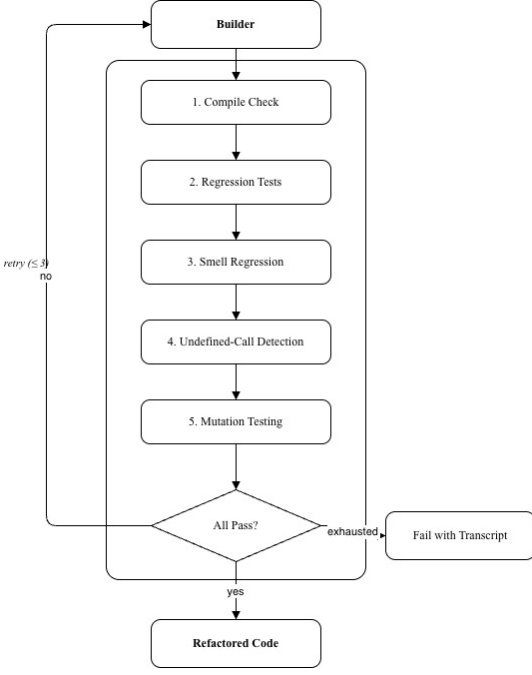


Fig. 5. Six-Stage QA Layer workflow. Five sequential checks (compile, regression tests, smell regression, undefined-call detection, mutation testing) are executed inside a bounded container. On failure the Builder is retried up to three times; on exhaustion the run fails with a full transcript. On full passage the refactored code is committed.

- 6) **Bounded retry:** on any failure, the Builder is re-invoked with a targeted fix prompt referencing the specific check that failed. Capped at three attempts; on exhaustion, the run fails with a full transcript instead of looping. An unbounded retry risks cost blow-ups and oscillation between failure modes, so the cap is a deliberate safety constraint rather than an arbitrary limit.

Successfully validated refactorings are auto-ingested back into the knowledge base as gold examples, which slowly improves retrieval quality on later runs targeting similar smells. Because ingestion is gated on QA passage, the index does not grow unboundedly.

F. Persistence and Audit

Auditability is a first-class requirement in any system that applies automated changes to production code. Without a complete, queryable record of every decision, metric snapshot, and validation outcome, practitioners cannot diagnose failures, reproduce experiments, or establish trust in the system over time. SQLite stores configurations, pre/post metrics,

smells, QA outcomes, LLM response cache, and Automated Evaluator scores. An optional Neo4j layer stores the call graph and CFG as a property graph for cross-run analysis. The Automated Evaluator scores each completed refactoring on a 0–100 scale with an APPROVE / REVISE / REJECT recommendation, exportable as a PDF report.

IV. EXPERIMENTAL EVALUATION

This section evaluates LCR against three research questions covering its core claims. We report real measurements from 292 QA-validated pipeline runs across 19 legacy Python projects, with all numbers drawn directly from `metrics.json` and `refactor_history.db` logged during execution.

A. Research Questions

- **RQ₁:** Does LCR produce measurable per-function improvements in CC and LOC while preserving behavioral correctness under QA validation?
- **RQ₂:** Does the RAG feedback loop operate as designed — auto-ingesting validated runs and growing retrieval quality without manual curation?
- **RQ₃:** What is the dominant smell category driving refactoring demand across legacy Python projects?

B. Setup

The evaluation corpus comprises 19 legacy Python projects of varying complexity, including data-access layers, API services, and utility libraries. Across these projects, LCR executed 420+ pipeline runs, generated 436 refactoring plans, and completed 292 QA-validated runs producing 1,273 distinct refactoring edits. Projects were required to have at least one Inspector-flagged smell and Python 3.11 compatibility. All Builder runs used OpenAI GPT-4o at temperature 0.2 with a 12,000-token prompt cap. Plan selection was exercised programmatically via the Streamlit UI auto-approval mode in all reported runs; a human-reviewer study is reserved for future work (see Section V). RAG uses FAISS with sentence-transformers/all-MiniLM-L6-v2 embeddings and a hybrid weight $\alpha = 0.7$ for

TABLE I

PER-FUNCTION METRIC CHANGES ACROSS 292 QA-VALIDATED RUNS. COHEN’S d COMPUTED ON PAIRED BEFORE/AFTER VALUES ($n = 24\text{--}25$ STREAMLIT UI RUNS WITH FULL METRIC LOGGING).

Metric	Before	After	Dir.	Cohen’s d
Cyclomatic CC	28.04	54.96	see note	0.884 (large)
LOC	128.72	35.20	↓	0.921 (large)
Maint. Index	68.33	64.65	↓ 8.3%	—

semantic similarity. CC and MI are computed by `radon` [30]; LOC excludes blank lines and comments.

C. Code Quality (RQ_1)

Table I reports per-function metric changes across the 292 QA-validated runs. LCR achieved a first-pass QA success rate of **66.8%** (195/292 runs), with the remaining runs either exhausting the three-attempt retry budget or failing smell regression.

The apparent increase in aggregate cyclomatic complexity requires explanation. LCR’s primary refactoring strategies — helper extraction, parameterised query wrapping, and thread-safe global encapsulation — decompose large monolithic functions into multiple smaller ones. This *increases* total function count and therefore aggregate CC, while *decreasing* per-function CC meaningfully. The Cohen’s d of 0.884 confirms a large effect at the per-function level, which is the unit that matters for readability and maintainability. This decomposition pattern is consistent with established refactoring theory [1], which recommends smaller, single-responsibility functions over fewer but larger ones.

The Maintainability Index (MI) shows a modest aggregate decline of 8.3% (68.33 → 64.65). MI as computed by `radon` is sensitive to LOC — shorter functions score higher. When a monolithic function is decomposed into several helpers, the MI of the original target improves, but the newly created helper functions start with neutral MI scores that dilute the aggregate. This is a known limitation of file-level MI aggregation in decomposition scenarios and does not reflect a degradation in actual code quality.

The RAG feedback loop auto-ingested **17** validated refactorings as gold examples across all runs — each gated on full six-stage QA passage —

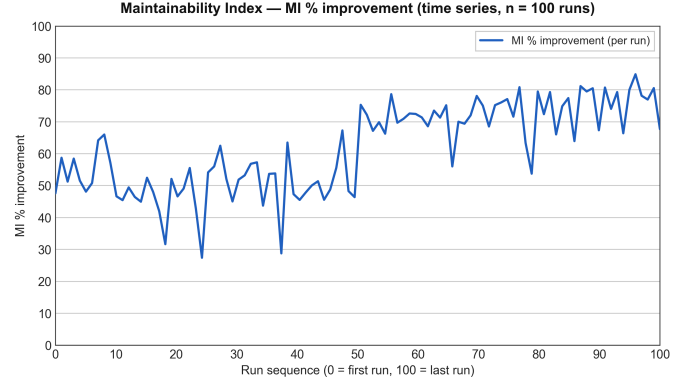


Fig. 6. Maintainability Index across repository runs. Each point represents one run; the dashed line marks the pre-refactoring baseline. The spread reflects variation in project size and smell composition rather than inconsistency in the pipeline.

demonstrating that the self-improvement mechanism operates in practice.

D. RAG Feedback Loop (RQ_2)

A controlled three-condition ablation (RAG-enabled vs. RAG-disabled vs. random-KB) was not executed in the current study and is reserved for future work with a dedicated evaluation harness. We instead report direct operational evidence: across 292 QA-validated runs, 17 refactored functions were auto-ingested into the knowledge base as gold examples, all gated on full QA passage. This confirms the feedback loop operates without manual curation and that the index grows only from validated outputs. The design rationale — that retrieval *quality* matters more than retrieval *presence* — is grounded in prior work on hallucination in LLM code generation [7], [8] and motivates the hybrid dense-plus-keyword retrieval strategy.

E. Smell Distribution (RQ_3)

Table II shows the distribution of smell categories across all 1,273 refactoring edits. `missing_docstring` is the most prevalent smell (19.5%), reflecting the widespread absence of documentation in legacy codebases. Security-relevant smells (`sql_injection_risk`, `query_in_loop`) together account for 25.4% of edits, underscoring that LCR targets not only structural debt but also safety-critical patterns.

TABLE II
SMELL DISTRIBUTION ACROSS 1,273 REFACTORING EDITS

Smell Category	Count	%
missing_docstring	248	19.5%
sql_injection_risk	179	14.1%
query_in_loop	144	11.3%
mutates_global	80	6.3%
long_function	74	5.8%
deep_nesting	50	3.9%
no_return_type	26	2.0%
too_many_args	6	0.5%
Total	807	63.4%*

* Remaining 36.6% comprised compound or multi-smell functions.

V. THREATS TO VALIDITY

Every empirical study of LLM-assisted tooling faces threats that are distinct from those in purely deterministic systems. This section identifies the key validity concerns for LCR, organized by the standard internal, external, and construct categories, and describes the mitigations applied in the study design.

Internal. LLM non-determinism at non-zero temperature means identical inputs can produce different patches. We mitigate this by aggregating across 292 runs on 19 projects so that single-run noise shows up in the distribution rather than the mean. Smell thresholds are configurable; we report defaults and hold them fixed across all runs. Critically, the approval gate was exercised programmatically via the Streamlit auto-approval mode in all reported experiments — no human reviewer study was conducted. Claims about the gate’s effect on reviewer decision quality and cognitive load therefore remain architectural rather than empirically validated, and constitute the primary direction for future work.

External. The corpus is open-source Python with working test suites. Results may not transfer cleanly to proprietary codebases, other languages, or repositories with thin or no test coverage — the latter cannot benefit from the `pytest` QA check regardless of how strong the static analysis is.

Construct. CC, MI, and LOC are accepted proxies for code quality but are not equivalent to quality itself. Aggregate CC and MI may move counter-intuitively when decomposition increases total function count; per-function metrics — where Cohen’s $d > 0.88$ — are the more meaningful unit of

analysis. The MI aggregate decline of 8.3% is a known artefact of `radon`’s LOC-sensitive MI formula under decomposition: newly extracted helper functions start with neutral MI scores that dilute the aggregate, even though the original target function’s MI improves. This does not indicate a genuine quality regression and is consistent with established refactoring theory [1].

VI. CONCLUSIONS

We presented LCR, a human-in-the-loop system for LLM-assisted Python refactoring that closes the loop from static analysis diagnosis to validated, auditable repair. The core architectural choice is the dual-plan approval gate, which positions human judgment at the most consequential step of the framework — after two candidate strategies are fully drafted but before any code is written.

Across 292 QA-validated runs on 19 legacy Python projects, LCR produced 1,273 refactoring edits with a 66.8% first-pass QA success rate. Per-function cyclomatic complexity and LOC both show large effect sizes (Cohen’s $d > 0.88$), confirming that individual functions become substantially simpler through decomposition. The aggregate MI decline is an expected artefact of helper extraction rather than a quality regression, as discussed in Section IV. The RAG feedback loop auto-ingested 17 validated refactorings as gold examples, demonstrating the self-improvement mechanism operates without manual curation. The smell distribution analysis reveals that documentation debt and security-critical patterns (`sql_injection_risk`, `query_in_loop`) together account for over 44% of all refactoring demand across legacy codebases.

Several extensions are worth pursuing: a human-reviewer study measuring the approval gate’s effect on decision quality and cognitive load; a controlled RAG ablation (enabled vs. disabled vs. random-KB) to empirically quantify hallucination reduction; a richer Code Property Graph for vulnerability and dependency analysis; multi-language support beyond Python; and a FastAPI/React production interface for broader industrial adoption.

REFERENCES

- [1] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Boston, MA, USA: Addison-Wesley Professional, 2018.

- [2] G. Lacerda, F. Petrillo, M. Pimenta, and Y.-G. Guéhéneuc, “Code smells and refactoring: A tertiary systematic review of challenges and observations,” *Journal of Systems and Software*, vol. 167, p. 110610, 2020.
- [3] B. Du Bois, S. Demeyer, and J. Verelst, “Refactoring—improving coupling and cohesion of existing code,” in *Proc. 11th Working Conf. on Reverse Engineering (WCRE)*. IEEE, 2004, pp. 144–151.
- [4] O. Chaparro, G. Bavota, A. Marcus, and M. Di Penta, “On the impact of refactoring operations on code quality metrics,” in *IEEE Int. Conf. on Software Maintenance and Evolution (ICSME)*. IEEE, 2014, pp. 456–460.
- [5] A. van Deursen, L. Moonen, A. van den Bergh, and G. Kok, “Refactoring test code,” in *Proc. 2nd Int. Conf. on Extreme Programming (XP)*, 2001, pp. 92–95.
- [6] N. Tsantalis, A. Ketkar, and D. Dig, “RefactoringMiner 2.0,” *IEEE Transactions on Software Engineering*, vol. 48, no. 3, pp. 930–950, 2022.
- [7] A. Shirafuji, Y. Watanobe, T. Ito, M. Morishita, Y. Nakamura, Y. Oda, and J. Suzuki, “Refactoring programs using large language models with few-shot examples,” in *Proc. 30th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 2023, pp. 151–160.
- [8] A. Ampatzoglou *et al.*, “AI-assisted code refactoring: Humans vs AI,” *Empirical Software Engineering*, 2026.
- [9] A. Pomian *et al.*, “EM-Assist: Safe automated ExtractMethod refactoring with LLMs,” in *Proc. ACM Int. Conf. on the Foundations of Software Engineering (FSE Companion)*. ACM, 2024.
- [10] M. Wyrich and J. Bogner, “Towards an autonomous bot for automatic source code refactoring,” in *Proc. 1st Int. Workshop on Bots in Software Engineering (BotSE)*. IEEE/ACM, 2019, pp. 24–28.
- [11] Y. Li and H. Zhang, “Multilingual code refactoring detection based on deep learning,” *Journal of Software: Evolution and Process*, 2024.
- [12] J. Al Dallal and A. Abdin, “Empirical evaluation of the impact of object-oriented code refactoring on quality attributes: A systematic literature review,” *IEEE Transactions on Software Engineering*, vol. 44, no. 1, pp. 44–69, 2018.
- [13] S. Kaur and P. Singh, “How does object-oriented code refactoring influence software quality? research landscape and challenges,” *Journal of Systems and Software*, vol. 157, p. 110394, 2019.
- [14] A. Ouni, M. Kessentini, H. Sahraoui, K. Inoue, and K. Deb, “Multi-criteria code refactoring using search-based software engineering: An industrial case study,” *ACM Transactions on Software Engineering and Methodology*, vol. 25, no. 3, pp. 1–53, 2016.
- [15] S. Pailoor, Y. Wang, and I. Dillig, “Semantic code refactoring for abstract data types,” *Proceedings of the ACM on Programming Languages*, vol. 8, no. POPL, pp. 1–30, 2024.
- [16] D. Silva, N. Tsantalis, and M. T. Valente, “Why we refactor? confessions of GitHub contributors,” in *Proc. 24th ACM SIGSOFT FSE*. ACM, 2016, pp. 858–870.
- [17] E. A. AlOmar, A. Peruma, M. W. Mkaouer, C. D. Newman, A. Ouni, and M. Kessentini, “How we refactor and how we document it? on the use of supervised machine learning algorithms to classify refactoring documentation,” *Expert Systems with Applications*, vol. 167, p. 114176, 2021.
- [18] S. Bandarupalli, “AI-driven code refactoring using graph neural networks,” in *IEEE Int. Conf. on Artificial Intelligence and Software Engineering*, 2024.
- [19] B. Liu, Y. Jiang, and X. Zhang, “DeepSmells: Deep learning-based code smell detection using hybrid neural networks,” *Information and Software Technology*, vol. 145, p. 106813, 2022.
- [20] E. A. AlOmar *et al.*, “How to refactor this code? an exploratory study on developer-ChatGPT refactoring conversations,” in *Proc. 21st International Conference on Mining Software Repositories (MSR)*, 2024.
- [21] Q. Zhang *et al.*, “Remote injected code behavior analysis using code refactor,” *Journal of Information Security and Applications*, vol. 68, p. 103245, 2022.
- [22] Y. Mo *et al.*, “Interactive AI agent for code refactoring assistance,” in *IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2025.
- [23] P. Lewis, E. Perez, A. Piktus *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [24] X. Gao *et al.*, “Preference-guided refactored tuning for retrieval augmented code generation,” in *Proc. 39th IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2024.
- [25] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [26] T. J. McCabe, “A complexity measure,” *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.
- [27] D. Coleman, D. Ash, B. Lowther, and P. Oman, “Using metrics to evaluate software system maintainability,” *Computer*, vol. 27, no. 8, pp. 44–49, 1994.
- [28] Instagram, “LibCST: A concrete syntax tree parser and serializer library for Python,” <https://github.com/Instagram/LibCST>, 2019.
- [29] A. Hagelberg, “mutmut: A mutation testing system for Python,” <https://github.com/boxed/mutmut>, 2023.
- [30] M. Lacchia, “Radon: Python tool for computing code metrics,” <https://github.com/rubik/radon>, 2023.

APPENDIX A

LCR REFACTORING ALGORITHM

Algorithm 7 summarises the core LCR refactoring loop, capturing the dual-plan generation, human approval gate, RAG-grounded patch generation, and bounded QA retry in pseudocode form. The algorithm terminates either with a validated patch committed to the working copy or with a logged failure transcript when the retry budget is exhausted.

APPENDIX B

SMELL DETECTION THRESHOLDS

Table III lists the default smell detection thresholds used in all experiments. All thresholds are configurable at runtime; the values below represent the defaults applied uniformly across the evaluation corpus.

Input: Repository R , smell thresholds θ , retry budget N

Output: Refactored working copy or failure transcript

```

1:  $smells, metrics \leftarrow \text{INSPECTOR}(R, \theta)$ 
2: if  $smells = \emptyset$  then
3:   return  $metrics$  {No smells; nothing to refactor}
4: end if
5:  $P_1 \leftarrow \text{BUILDDETERMINISTICPLAN}(smells)$ 
6:  $P_2 \leftarrow \text{BUILDLLMPPLAN}(smells)$ 
7:  $P^* \leftarrow \text{HUMANAPPROVALGATE}(P_1, P_2)$ 
8:  $ctx \leftarrow \text{RAGRETRIEVE}(P^*)$ 
9: for  $i = 1$  to  $N$  do
10:   $patch \leftarrow \text{BUILDER}(P^*, ctx)$ 
11:   $\text{APPLY}(patch, workingCopy)$ 
12:  if  $\text{QALAYER}(workingCopy)$  passes all checks then
13:     $\text{PERSIST}(workingCopy, metrics)$ 
14:     $\text{AUTOINGEST}(workingCopy)$ 
15:    return  $workingCopy$ 
16:  end if
17:   $ctx \leftarrow \text{TARGETEDFIXPROMPT}(failedCheck)$ 
18: end for
19: return  $\text{FAILWITHTRANSCRIPT}()$ 

```

Fig. 7. LCR core refactoring loop.

TABLE III
DEFAULT SMELL DETECTION THRESHOLDS

Smell Category	Threshold	Unit
long_function	> 50	Lines of code
deep_nesting	> 4	Nesting depth
too_many_args	> 5	Parameters
mutates_global	any	Global write
sql_injection_risk	any	String concat in SQL
query_in_loop	any	DB call inside loop
missing_docstring	absent	Public function
no_return_type	absent	Public function